

Acquisition and Analysis of Neural Data

Lecture Notes and Python Analysis Workspace

Arjun Puri

Bernstein Center for Computational Neuroscience, Berlin

Summer 2026

Contents

Preface	v
Notation	vii
I Foundations	1
1 Course Overview and Data Analysis Workflow	3
1.1 Analysis philosophy	3
1.2 Canonical analysis loop	4
1.3 Reusable computation	4
1.4 Next steps	4
Bibliography	7

Preface

These notes collect the mathematical and practical ideas needed to turn neural recordings into interpretable data objects. The emphasis is on spike trains, trial structure, summary statistics, and the assumptions that connect raw measurements to models of neural coding.

The notes are organized so that conceptual exposition, derivations, and reproducible analysis decisions stay in the same place. Code can reproduce figures and explore examples, but the notes state the definitions, conventions, and interpretation rules that make those analyses scientifically meaningful.

The first chapter sets the workflow: start from a recording, choose a response representation, quantify structure with simple statistics, and only then fit richer encoding or decoding models. This order is intentional. In neural data analysis, a poorly defined response variable cannot be rescued by a sophisticated downstream model.

Berlin, April 2026

Notation

Spike-train notation

Symbol	Meaning
t_i	Time of spike i
$\rho(t)$	Spike-train response function
$r(t)$	Estimated firing-rate curve
Δt	Temporal bin width or smoothing scale

Probability and statistics

Symbol	Meaning
$p(x)$	Probability density or mass function
$\mathbb{E}[\cdot]$	Expectation
$\text{Var}[\cdot]$	Variance
$C_{xy}(\tau)$	Cross-covariance at lag τ

Linear algebra

Symbol	Meaning
$\mathbf{x}$	Column vector
$\mathbf{X}$	Matrix
$\text{tr}(\mathbf{X})$	Trace of a matrix
$\text{diag}(\cdot)$	Diagonal operator

Part I

Foundations

1

Course Overview and Data Analysis Workflow

Acquiring neural data is only the first step. Once an experiment has produced spike trains, local field potentials, calcium traces, or behavior-aligned recordings, the real work begins: organizing the data, defining the response variables, choosing sensible summary statistics, and building analyses that survive contact with noise and biological variability.

The central workflow is not “collect data, then run a model.” It is a sequence of modeling decisions: what counts as a response, which time origin matters, which variability is signal rather than nuisance, and which assumptions justify the statistics being reported.

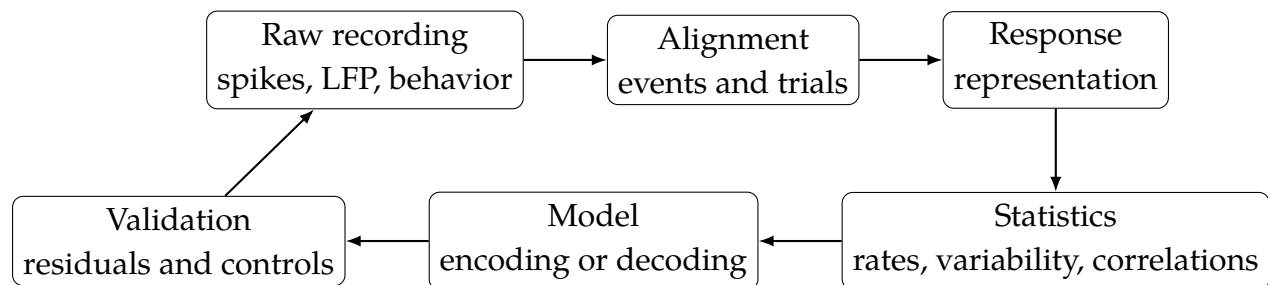


Figure 1.1: A disciplined neural-data workflow is cyclic. Validation often sends the analysis back to earlier choices about alignment, representation, or summary statistics.

1.1 Analysis philosophy

The guiding principle is that prose and computation should reinforce each other. A derivation should make clear what an estimator means, and code should make the estimator’s behavior visible on simulated or real data. When a preprocessing choice changes the scientific interpretation, the notes should say so explicitly rather than hiding the choice inside a script.

Core workflow

Start from raw measurements, define the neural response representation, build summary statistics, and only then fit heavier models. Clean abstractions early make later inference much easier.

1.2 Canonical analysis loop

A useful first-pass workflow for many neural datasets is:

1. load and validate raw data,
2. align signals to task-relevant events,
3. compute simple descriptive statistics,
4. visualize structure before fitting models,
5. and keep the full pipeline reproducible.

This sequence sounds obvious, but it guards against many common mistakes. It is often more informative to examine rasters, marginal distributions, and trial averages before building a large decoding model. Classic computational-neuroscience texts repeatedly emphasize that representation choices shape every downstream conclusion [DA01; Ger+14; PPL07].

Canonical rate estimate

$$r(t) = \sum_{i=1}^n w(t - t_i)$$

Here w is a smoothing kernel and t_i are spike times. This simple construction turns a point process into a continuous signal that can be visualized, compared across conditions, and passed into downstream models.

1.3 Reusable computation

Reusable code should live at the level of concepts, not one-off figures. Natural expansion points are dataset loaders, event-alignment helpers, spike-train feature extraction, plotting utilities for rasters and correlograms, and reproducible studies that document each analysis decision. The goal is not to automate interpretation away; it is to make the mechanical parts reliable enough that the scientific assumptions are exposed.

1.4 Next steps

As the notes grow, each lecture block should end with three things: the mathematical object introduced, the empirical diagnostic that estimates it, and the failure mode to check before

trusting it. That convention keeps the course focused on analysis rather than notation alone.

Bibliography

- [DA01] Peter Dayan and L. F. Abbott. *Theoretical Neuroscience*. MIT Press, 2001.
- [Ger+14] Wulfram Gerstner, Werner M. Kistler, Richard Naud, and Liam Paninski. *Neuronal Dynamics*. Cambridge University Press, 2014.
- [PPL07] Liam Paninski, Jonathan Pillow, and Jeremy Lewi. “Statistical models for neural encoding, decoding, and optimal stimulus design”. In: *Progress in Brain Research* 165 (2007), pp. 493–507.